

Introduction to Programming II

project log

Project title: [DoodleLab.com](https://doodlelab.com) - Drawing App (Check-in: January 23, 2026)

Topic: Extending stampTool.js – Shape Stamping + UI/Styling
Fixes: Shape stamping functionality (image + geometric stamps), fixing layout/styling issues (sidebar and theme).

What progress have you made on this topic?

This week I focused on extending the stamp tool in my drawing application by allowing it to support both image stamps and shape stamps. The original template only allowed the user to place preloaded image stickers on the canvas. My goal was to extend this so that the stamp tool could also generate shapes dynamically.

To implement this, I created a type selector dropdown inside the populateOptions() function of stampTool.js. This dropdown allows the user to choose between several stamp types:

- Image (original sticker stamps)
- Triangle
- Square
- Rectangle

I added the following variables to support this system:

```
var stampTypeSelect;  
var stampSelect;  
var stampType = 'image';
```

The dropdown was created using the createSelect() function from p5.js and attached to the .options panel. I also added logic so that when the user selects Image, the existing image selector remains visible, but when a shape is selected the image selector hides automatically. This allows the same tool to support both image stamps and generated shapes without cluttering the interface. Inside the draw() function, I implemented conditional logic to render the correct stamp type at the mouse position. When the stamp type is set to image, the tool uses the existing image stamping logic with tint() for opacity control. When a shape type is selected, the tool instead draws shapes using p5.js functions such as triangle(), rect(), and ellipse(). Each shape uses the currently selected colour from the application's colour palette by referencing: colorP.selectedColour This allowed shapes to behave consistently with the rest of the drawing tools.

After confirming the basic shape system worked, I extended the tool further by adding additional shapes:

- Circle
- Star
- Octagon

These shapes required additional geometry logic. For example, the star and octagon were drawn using beginShape() and vertex() loops combined with trigonometry (cos() and sin()) to calculate each vertex. All stamps—both images and shapes—respect the existing size slider and opacity slider, so the new functionality integrates with the original stamp controls instead of introducing additional UI elements.

What problems have you faced and were you able to solve them?

While implementing the new stamp types, I encountered several issues that required debugging.

The first major issue was that all stamp functionality stopped working entirely, including the original image stamps. When I tried to use the stamp tool, none of the shapes or images appeared.

After reviewing the code I realised that the stampSelect variable had been declared inside the populateOptions() function using let, which meant it was not accessible inside the callback used by the dropdown change event. This created a scope problem where the callback could not properly reference the image selector.

I fixed this by declaring the variable at the top level using var so that it remained accessible throughout the tool: var stampSelect;

I also discovered that I had initially referenced a colour variable called fillColor, but the application actually stores the selected colour in colorP.selectedColour. Updating the code to use this correct variable allowed the shapes to use the same palette system as the rest of the drawing tools.

After fixing this, a second issue appeared where all drawing tools stopped working after the stamp tool was used. The browser console showed the following error: stampTool.js:93 Uncaught ReferenceError: square is not defined This error occurred because I used the square() function, which is only available in newer versions of p5.js. The version used in this project does not support that function. Because the error occurred inside the draw loop, it prevented the rest of the tools from running.

I fixed this by replacing the square() call with: rect(mouseX, mouseY, size, size); Using equal width and height produces the same visual result while maintaining compatibility with the p5.js version used in the project.

What are you planning to do over the next few weeks?

Over the next few weeks I plan to continue expanding the drawing tools and improving the usability of the application. My next priorities include improving the functionality of existing tools, adding more interactive features, and refining the user interface. I also want to continue improving the structure of the application by keeping the tool code modular and ensuring that new features integrate properly with the existing control systems such as colour selection, size controls, and opacity settings. In addition, I plan to spend more time testing the application to identify edge cases and unexpected behaviours that may occur when switching between tools.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

At this stage I believe I am on track to successfully complete the project. The core drawing functionality is working and I have already begun extending the template with additional features such as the expanded stamp tool and improved interface styling.

Although I encountered several issues while implementing these changes, I was able to resolve them through debugging and reviewing the relevant code files. These problems also helped me better understand how different parts of the application interact, particularly how tool scripts integrate with the main drawing loop and UI structure. Going forward, my main focus will be on continuing to build features while making sure the application remains stable and easy to use.

Introduction to Programming II

project log

Project title:	DoodleLab.com - Drawing App (Check-in: January 30th, 2026)
Topic:	Creating UndoRedo.js – Undo/Redo Functionality + UI/Styling Fixes (Check-in: January 23, 2026): Undo/Redo functionality (for all actions) + fixing layout/styling issues (sidebar + theme).
What progress have you made on this topic?	

This week I focused on redesigning how tool options are displayed in the drawing app. Originally, the app used a bottom .options area, but I wanted a more structured interface where clicking a tool would open a rectangular side panel extending from the toolbox. The idea was for this panel to span the usable height of the screen, display that tool's settings, and include a collapse button on the outer right edge so the user could close it when finished.

To build this, I added a new toolPanel structure in index.html: `<div id="toolPanel">
 <button id="collapseBtn"></button>
 <div id="toolPanelContent"></div>
</div>`

I then updated toolbox.js so that when a tool is clicked, the panel opens by adding the open class to #toolPanel. I also restored a light background highlight on the selected sidebar tool so there was still a clear active-state indicator. Inside the same selection logic, I added code to populate the panel with placeholder content and later with the tool's actual controls.

A major part of this week's progress was moving away from the old .options system. I changed tool option rendering so it would appear inside #toolPanelContent instead. I started with stampTool.js, then realised I needed to update the other tools as well. That meant changing selectors in:

- freehandTool.js
- LineToTool.js
- sprayCanTool.js
- mirrorDrawTool.js

For example, I replaced code like: `opacitySlider.parent(select(".options"))`, with: `opacitySlider.parent(select("#toolPanelContent"))`;

I also removed the old .options div from index.html and simplified the CSS grid layout from three columns to two. Since the bottom options area was gone, I reworked the page layout so the colour palette stays fixed at the bottom of the screen and the tool panel stretches between the header and the palette.

To make the layout more responsive, I changed the wrapper height to 100vh, fixed the colour palette to the bottom, and updated the panel to use top and bottom positioning instead of relying on a fixed height calculation. This made the panel behave more consistently across screen sizes.

I also started adding undo/redo functionality during this stage. I created logic in undoRedo.js to store previous canvas states and began testing how drawing actions should be grouped so that undo would reverse a full action rather than a single frame.

What problems have you faced and were you able to solve them?

The first problem I ran into was with the collapse button. The panel would open correctly when a tool was clicked, but the collapse button would either look misaligned or not work at all. I adjusted the CSS positioning several times, including changing the transform and right values, which improved the visual placement, but the button still was not functioning properly. At first I attached the collapse logic using `window.addEventListener('load', ...)`, expecting the listener to connect once the page loaded. To debug it, I added several `console.log()` statements, including messages inside the load handler and inside the click handler. The important clue was that only my general "toolbox running" message appeared, while none of the logs inside the `window.load` callback appeared. That told me the event listener setup was not happening the way I expected. Because of that, I removed the `window.load` approach and moved the collapse button setup directly into the `toolbarItemClick` flow.

I also added a short timeout to ensure the button existed before attaching the listener. This worked better because it tied the button logic to the moment the panel was actually being used rather than relying on a page-load setup that was not behaving correctly. Another problem appeared after I removed the old `.options` area. Once I started selecting tools, the app threw this error: `freehandTool.js:17 Uncaught TypeError: Cannot read properties of null (reading 'html')`.

This happened because some tools were still trying to populate `.options`, even though that element no longer existed in the HTML. I solved this by updating all remaining tools to use `#toolPanelContent` instead. Once all tools were pointing to the new container, their controls began rendering correctly inside the side panel.

I also had a smaller interface issue in `stampTool.js` where the type dropdown appeared above the label instead of below it. That turned out to be caused by the order in which I was creating the label and select element. Reordering those elements fixed the panel layout.

The biggest technical problem this week came from the undo system. At first, undo did not work correctly for the freehand tool. In some cases it did nothing, and in others it would undo tiny parts of a stroke frame by frame instead of treating the whole mouse action as one drawing event. I traced this back to the fact that `saveState()` was being called too often during drawing. I first tried introducing an `isDrawing` flag so that the state would only be saved before drawing started. That improved the logic, but it still did not fully solve the issue. I then changed the logic again by using a `hasDrawn` flag so I could distinguish between simply clicking and actually drawing a line.

Even after that, undo still behaved incorrectly. I eventually realised the main problem was in the undo stack logic itself. The app was restoring the state that had just been popped off the stack, which meant it was redrawing the current state instead of stepping back to the previous one. I fixed this by changing the undo function so it:

1. Pops the current state
2. Pushes that state onto the redo stack
3. Restores the previous state still left on the undo stack

That was the point where the undo behaviour started making sense. Instead of displaying the drawing again, it finally stepped back properly to the state before it.

What are you planning to do over the next few weeks?

I plan to keep refining both the tool panel system and the undo/redo behaviour so they feel reliable across all tools. This phase made it clear that moving UI elements is not just a styling task, it also affects how tools populate controls and how the whole layout behaves.

My next step is to make sure every tool fully integrates with the side panel cleanly, without leftover dependencies on the original template structure. I also want to continue improving the spacing and styling of controls inside the panel so the interface feels more intentional and easier to use.

On the functionality side, I want to keep testing undo/redo with different tools and interaction patterns to make sure actions are saved at the correct moment. I also plan to focus more on testing and interface debugging, because I am starting to move out of the stage of just adding features and more into the stage of making the whole app feel stable and consistent.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

I believe I am on target to complete the project successfully, although this week showed me that some structural changes take longer than they first appear. What started as a layout improvement ended up requiring changes across multiple tools, the HTML structure, and the CSS grid system.

Even though that slowed things down, I do not think I am off track. The changes I made this week were important because they improve the long-term structure of the application. Instead of keeping the original template layout and forcing new features into it, I have started reshaping the app so it can support more complex tools in a cleaner way.

The main way I will stay on track is by continuing to work in a more modular way, testing changes as I go, and focusing less on adding lots of new tools at once and more on making sure the existing systems behave properly. That should help me avoid creating more bugs while still making progress.

Introduction to Programming II

project log - February 6

Project title:	DoodleLab.com - Drawing App (Check-in: February 6th, 2026)
Topic:	Creating Project save/load, and implementing File API's + image scaling. Adjusting Color palette function: Save images created to the image gallery for reuse, Fixing layout/styling issues (sidebar + theme), removing the colour palette icon and creating a fixed floating colour palette

What progress have you made on this topic?

During this stage of development I focused on extending the drawing application's functionality while improving the overall software architecture and user interface. A major improvement was the implementation of a state persistence system for saving canvas backgrounds. I created a new module called stateSaving.js, which serializes the current canvas state using the p5.js get() function and converts it into a base64 encoded data URL. These serialized states are then stored in localStorage, allowing background images to persist between browser sessions.

To maintain a modular architecture, this functionality was implemented as a toolbox tool rather than a standalone UI element. The tool uses the existing toolbox structure and provides a panel interface containing buttons for saving the background and displaying preview thumbnails of previously saved states. Each preview is clickable, which allows the user to restore the selected background. Integrating the feature into the toolbox system ensured consistency with the application's tool-based interaction model.

Another major improvement involved refactoring the colour palette system. The original implementation relied on static palette behaviour attached to an icon. I redesigned this into a context-sensitive floating control panel that appears only when colour-dependent tools are selected (such as Freehand, SprayCanTool, and MirrorDrawTool). The panel contains three RGB sliders that update a preview box in real time. The sliders dynamically update their gradient backgrounds to reflect the current colour range, providing immediate visual feedback to the user.

I also implemented preset colour swatches that allow users to quickly select commonly used colours. These presets are integrated into the same colour control panel so that the entire colour system remains visually grouped.

Another interface improvement involved redesigning the stamp selection system. The original dropdown menu was replaced with a grid-based preview interface, allowing users to visually select stamps. This improves usability by reducing cognitive load and making the interface behave more like professional drawing tools.

Finally, I strengthened the undo/redo system integration. Each drawing tool now calls saveState() after completing its operation so that canvas states are pushed to the undo stack. This ensures that the undo system can reliably revert previous drawing operations.

Overall, this phase significantly improved the modularity, usability, and reliability of the drawing application.

What problems have you faced and were you able to solve them?

One issue involved undo history not recording drawing operations. Although the `undoRedo.js` file correctly implemented an undo stack, some tools were not calling the `saveState()` function after completing drawing actions. This prevented the undo system from tracking these operations. I resolved this by updating several tools (`LineToTool`, `SprayCanTool`, `MirrorDrawTool`, and `StampTool`) so that each tool calls `saveState()` once the drawing action finishes.

Another issue occurred when modifying the interface and removing header buttons. Some scripts still attempted to attach event listeners to deleted elements, which resulted in the console error:

Uncaught TypeError: Cannot read properties of null (reading 'mouseClicked')
at new HelperFunctions (helperFunctions.js:18:28). This error occurred because the script attempted to bind a click handler to a button that had been removed from the DOM. I fixed this by removing the obsolete event listener.

What are you planning to do over the next few weeks?

Over the next few weeks I want to continue building on this by improving both the tool feedback systems and the state management features.

For the interface side, I want to keep refining the tool panel so that the controls feel more intentional and consistent across different tools. The freehand preview worked well as a first step, so I want to think more carefully about where preview elements are useful and where they would just create clutter.

For the background saving system, I want to keep testing how it behaves alongside the undo/redo system and make sure saved states restore cleanly in different situations. I also want to improve how saved backgrounds are selected so that the process feels clearer and more visual.

More broadly, I am beginning to move away from just adding tools and into a phase that is more focused on testing, interface design, and debugging unexpected behaviour. At this point, making the app feel reliable is becoming just as important as adding new features.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

This week's work was important because it did not just add features for the sake of adding more. It improved the way the application behaves for the user and also improved the internal structure of the project. The brush preview makes the freehand tool more understandable before drawing begins, and the state saving system adds a practical feature that makes the canvas more reusable over time.

I did run into some frustrating issues, especially with tools drawing behind the panel, but I was able to work through them by testing different possibilities, discarding fixes that caused new problems, and then narrowing the issue down properly. That process slowed me down at points, but it also helped me understand the interaction between `p5.js` controls, mouse input, and tool behaviour much better.

At this stage I am still on track, but my focus now needs to be on keeping the app stable while I continue refining it. The best way for me to stay on target is to keep testing each new change carefully and avoid overloading one file or feature with too many responsibilities.

Introduction to Programming II

project log

Project title:	DoodleLab.com - Drawing App (Check-in: February 14th, 2026)
-----------------------	---

Topic:	Continuing to extend stateSaving.js panel for API Giphy Search: Giphy API Utilization, SEO for selective Giphy rendering to canvas,
---------------	---

What progress have you made on this topic?

In this phase of the project, I made strong progress in extending the image-search functionality of my drawing app. I first explored how to add a search bar inside the stateSaving.js panel, including the use of search.png as a visual icon and an input area where the user could type a search term.

From there, I connected the search bar to the logic in imageApiTool.js, which allowed searched Giphy results to be fetched dynamically through the API. Initially, the search returned a single random image that could be placed onto the canvas. Once that worked, I expanded the functionality so that instead of only one result, a preview grid of multiple images appeared beneath the search bar. I then increased the result count so that up to 50 images could appear when available.

A major part of my progress was also architectural. Rather than keeping this functionality inside stateSaving.js, I decided it would be cleaner and more user friendly to move it into its own toolbar tool using imageApiTool.js. I transformed that file into a complete tool with its own icon, search bar, Giphy integration, preview grid, and image placement controls. I then added search.png to the toolbar so the feature could be utilized independently.

Another important improvement I made was refining how the Giphy preview grid behaves after image selection. Originally, when a user searched for a Giphy and clicked one of the previewed images, the preview grid disappeared. I changed this so that the Giphy results remain visible even after an image is selected. Now, the previewed GIFs stay on screen until the user performs a completely new search. This made the tool feel much more usable because the user can continue browsing the same search results and choose a different image without needing to retype the same search term again.

To achieve this, I updated the logic in imageApiTool.js so the returned search results are cached instead of being lost when the panel refreshes. I also removed the part of the code that unnecessarily rebuilt the panel after an image was selected. This improved both the user experience and the internal structure of the feature.

Overall, I improved both the functionality and the structure of the application. The user can now click the search tool, type a term, preview multiple Giphy images, select one, and place it onto the canvas with existing controls such as opacity and size.

What problems have you faced and were you able to solve them?

I faced several problems during this stage, but I was able to solve most of them through step-by-step debugging and revision. One of the first issues was that the changes I expected to see were not appearing in the browser. This turned out to be related to editing the wrong file path, so I had to verify which version of stateSaving.js was actually being used by the app.

I also had layout issues. The search bar overlapped with the “Choose File” area, the image previews appeared underneath the toolbar, the search bar touched the right border of the panel, and the image grid displayed too close to the search field. I resolved these by adjusting margins, padding, container widths, and by creating a separate scrollable gridContainer for the preview images.

Another important problem was a runtime error in imageApiTool.js:

Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')

This happened because the file still contained old DOM event-listener code that was trying to target HTML elements that no longer existed. I solved this by removing the outdated code and consolidating the search behavior into the new tool workflow.

I also had to fix a visual regression where the search icon disappeared from inside the search bar after layout changes. That issue came from positioning rules, so I restored position: relative on the search container so the icon could sit correctly inside the input area again.

I also had several layout issues with the search results grid. At one stage, the images were appearing too close to the search bar, and at another stage the search bar was overlapping the pictures. I tried adjusting the grid container with margin-top, but that moved the search bar itself rather than actually creating the spacing I wanted. After testing a few variations, I realised the better fix was to increase the bottom margin on the search container instead of trying to push the grid down from its own side. That created proper separation between the bar and the preview images.

There was also a smaller issue where removing position: relative from the search container caused the search icon to stop appearing inside the search bar. I fixed that by adding position: relative back to the container, which allowed the absolutely positioned icon to sit correctly inside the input area again.

I solved this by introducing a cachedImages variable to store the current search results while also calling self.populateOptions() to have the opacity and size controls appear when a user selects an image, and added logic so cached search results could be restored when needed. This meant that once a user searched for a term, the preview grid stayed visible until they typed a new search query. This was an important fix because it made the feature feel more stable, consistent, and user-friendly.

What are you planning to do over the next few weeks?

I will continue refining the image search tool so that it feels stable and properly integrated with the rest of the application.

Now that the search logic has been separated into imageApiTool.js, I want to keep improving how the preview grid behaves, how selected images are loaded, and how smoothly the placement controls work after an image is chosen. I also want to continue improving the spacing and layout inside the tool panel so that the search tool feels polished rather than just functional.

More broadly, this stage of the project is making it clear that I need to spend more time on testing, UI refinement, and debugging unexpected behaviours, rather than only focusing on adding more features. I want to make sure the tools I already have are structured well and do not create layout or interaction issues elsewhere in the app.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

This week's work was important because it forced me to improve the structure of the project by separating out functionality that no longer belonged in the same file. Even though that restructuring took extra time, it was worth doing because it made the codebase cleaner and easier to reason about.

The main lesson from this stage was that adding a feature is not just about making it work once, it also needs to fit the architecture of the rest of the project. Moving the Giphy functionality into its own tool was a better long-term decision than continuing to force it into `stateSaving.js`.

At this point I still feel on track, but I know I need to keep being careful about file organisation and interface testing. The best way for me to stay on target is to continue refining features in a modular way and make sure each new addition fits the overall structure of the app rather than making one file handle too many unrelated jobs.

Introduction to Programming II

project log

Project title:	DoodleApp.com - Drawing App (Check-in: February 25th, 2026)
-----------------------	---

Topic:	Implementing cursor preview (brush size), customized brush strokes for the freehandTool, extending the sprayCanTool for adjustable sliders (speed, radius). UI/Styling Fixes and Adjustments, Testing interactions between multiple tools and ensuring that switching tools does not introduce unintended behaviour
---------------	---

What progress have you made on this topic?

This week I made major progress on expanding the freehand tool, improving the spray can tool, and building out the eraser/cut tool into something much more functional than the original template.

The biggest area of progress was the freehand tool. I started by adding a grid of stroke-type icons from the freehandStrokeIcons folder into the tool panel. Each icon acts as a button, allowing the user to switch between different stroke styles. I kept the styling exactly as I wanted visually and focused only on the logic behind the buttons. I also changed the default selected stroke to marker, and made sure the existing freehand drawing logic only runs when the marker icon is selected.

From there, I built several custom stroke types, each with its own rendering behaviour:

1. Marker – rectangular stroke with a soft bleed effect and semi-transparent buildup
2. Pencil – grainy, lighter graphite-style stroke using many small low-opacity particles
3. Pen – smooth, precise, solid line with clean rounded edges
4. Crayon – chunky, waxy texture with irregular coverage and heavier buildup
5. Paintbrush – watercolor-inspired stroke with softer layering and more fluid movement
6. Thick wet paint – dense, messy paint effect with splat-like marks and drag trails

A big part of this process was repeatedly refining each brush so it looked visually distinct. For example, the first pencil version used soft rectangles, but it did not really look like pencil, so I changed it to use smaller circular particles with lower opacity and tighter spacing. I also changed the pen after the first glossy version looked off, simplifying it into a cleaner and more realistic ballpoint-style stroke. I then adjusted the marker so it no longer looked too similar to the pen by making it rectangular and giving it more bleed.

I also improved the spray can tool. I made the output much more speckled by increasing the particle count into the hundreds, reducing dot size, and using a more natural radial distribution so the spray spreads outward more like real paint. I added a radius slider so the user can control how far the spray spreads, and the tool also includes a speed slider to control particle count and density.

Another important piece of progress was improving the eraser tool. I implemented a rectangular selection system where the user can drag across the canvas to select an area, preview the boundary, and then erase that section. I also created a mode-selection grid with icons so the user must choose which edit behaviour to use. The first mode was scissor/cut, and later I added an erase mode using a circular white ellipse brush. I then added an eraser size slider and a live preview circle beside it, similar to the freehand tool preview.

Finally, I improved the undo/redo integration for tools like the spray can so that those actions are now captured properly in history.

What problems have you faced and were you able to solve them?

The biggest challenge this week was that many of these features did not look right on the first try, even when they technically worked.

A good example of this was the pencil stroke. My first version used grainy soft rectangles, but visually it felt off and did not really resemble pencil on paper. I reworked it so it used smaller circular particles, lower opacity, tighter spacing, and more overlap. That made it feel much closer to a graphite texture. I had a similar issue with the pen stroke. My first version included a glossy highlight effect, but it looked too artificial, so I removed that and simplified the stroke into something cleaner and more believable.

I also had to solve the problem of the pen and marker looking too similar. Once both existed, it became clear the marker needed more personality. I fixed that by changing the marker into a more rectangular chisel-style stroke with semi-transparent buildup and additional bleeding around the edges. That helped separate it visually from the crisp pen stroke.

The paintbrush and wet paint styles also took several rounds of adjustment. The first watercolor-style brush looked too spaced out and random. I wanted it to look more like the movement of water, so I made it denser and more cohesive by adding a stronger core, increasing the number of particles, and tightening the spacing between them. The wet paint brush also went through a lot of experimentation. I adjusted the splats to become more elongated, then changed the whole stroke so it was composed of splat-like shapes, added drag trails, increased density, and later changed the orientation so shapes respond to mouse direction. I also tested extra lines and highlights, then removed parts that made it look less convincing. That process helped me shape the wet paint tool into something much more distinct.

On the spray can tool, the main issue was that the / functionality was not working for spray drawings even though it worked for other tools. After comparing it to the freehand tool, I realised the spray can was not saving its state after drawing finished. I fixed that by tracking whether drawing had occurred and then calling `loadPixels()` and `saveState()` when the mouse was released. That made the spray can tool behave properly with the history system.

A large amount of debugging this week also happened in the eraser tool. I ran into several issues while trying to create a rectangular selection preview. At first the orange border would not appear, then it appeared but stayed on the canvas and created multiple lines as I dragged. I eventually solved this using the `loadPixels()` / `updatePixels()` pattern so the border could act as a temporary preview rather than becoming permanently drawn onto the canvas. I also had to ensure that when the mouse is released, the border disappears before the actual erase action is saved. That took several rounds of testing and adjustment.

So overall, yes, I was able to solve the main problems this week, although many of them required several iterations rather than a one-step fix.

What are you planning to do over the next few weeks?

I will spend more time refining the quality, consistency, and usability of the ones I already have.

For the freehand tool, I want to keep testing the custom stroke types side by side so that each one feels clearly different and intentional rather than just being a variation of the same effect. The marker, pen, pencil, crayon, watercolor brush, and wet paint brush now all exist, but I still want to keep improving how natural they feel when actually used over longer drawing sessions.

I also want to continue refining the eraser/cut tool, especially now that it has multiple modes. That includes testing edge cases, checking how the selection preview behaves, and making sure the tool always works cleanly with the undo/redo system.

More broadly, I am entering the phase of the project where the focus needs to shift more toward testing, UI design, and debugging unexpected behaviours. At this point I do not want to just keep adding features without stabilising them. My goal is to make the tools feel more polished and reliable as a complete application.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

I still feel confident about the project overall and the progress im making, but the most important thing now is making sure the features I have built remain stable, consistent, and well integrated with the rest of the app. This week showed me that even when a feature technically works, it often still needs several passes before it looks and behaves the way I want. Because of that, my plan for staying on target is to focus more on refinement and testing rather than constantly expanding the toolset.

Introduction to Programming II

project log

Project title:	DoodleApp.com - Drawing App (Check-in: March 5th, 2026)
Topic:	Drawing App UI/UX Refinement, Responsive Layout, Tooltip Positioning, and Preview Controls: Interface styling, responsive design for Tablet, Mobile and Website, control previews, tooltip behaviour for keyboard shortcuts, and structural layout fixes overlapping issues
What progress have you made on this topic?	

During this stage of the project I focused mainly on user interface refinement, styling, and usability improvements across the drawing application rather than adding completely new tools. By this point most of the core functionality was already implemented, so my goal was to make the interface cleaner, more intuitive, and more visually consistent. One of the first improvements I made was adjusting the undo and redo buttons in the header. Originally the icons appeared too dark and slightly difficult to see against the background. I modified the CSS so the PNG icons have 70% opacity by default and become fully opaque on hover, which provides clearer visual feedback to the user. I also used a CSS filter to make the icons appear white so they better match the overall UI style. I also implemented a custom font for the application header. I downloaded the *GeistMono* font and placed the .ttf file inside an assets/fonts directory. I then loaded the font in sketch.js using the p5.js preload() function: `let headerFont; function preload() {headerFont = loadFont("assets/fonts/GeistMonoFont.ttf");}` After loading the font, I updated the CSS so the header text and the RGB labels in the color slider use the same font-family. This helped make the UI feel more consistent. I also improved the styling of multiple interface elements. The clear button, undo/redo buttons, and tool panel buttons were redesigned with semi-transparent backgrounds, rounded corners, and hover transitions so they all follow a consistent visual style. I applied similar styling rules to buttons inside panels such as the state saving controls and mirror tool controls so they share the same appearance. Another significant improvement was adding preview indicators for tool sliders. I extended the thickness preview concept so that all tools that use thickness sliders display a circular preview beside the slider showing the current brush size. I also added opacity preview circles beside opacity sliders. These circles dynamically update using `rgba()` values so users can see how transparent their stroke will be before drawing. I also implemented tooltip bubbles for the undo and redo buttons. These appear when the user hovers over the buttons and display keyboard shortcuts such as *Ctrl + Z* and *Ctrl + Shift + Z*. Creating these tooltips required adding additional HTML elements, CSS styling for the floating bubble, and a small JavaScript script to dynamically position the tooltip beneath the buttons. Another major UI change involved the collapse button for the tool panel. Originally the button was positioned inside the panel using absolute positioning, which caused it to be clipped by the panel edge. I solved this by moving the button outside the panel in the HTML and positioning it independently using `position: fixed`. I then redesigned the button into a pill-shaped element with rounded ends and a layered shadow, which makes it look like it sits above the panel rather than being embedded in it. Finally, I added several smaller interface improvements including: responsive CSS media queries for tablet and smaller screen sizes, hover scaling effects for stamp icons in the stamp tool grid, smaller and more compact undo/redo button dimensions, repositioning of certain UI controls such as the mirror tool direction button so it appears below the sliders in a more logical order. Overall, the progress this week focused on improving the visual clarity, responsiveness, and usability of the application.

What problems have you faced and were you able to solve them?

A major challenge during this phase involved tooltip positioning for the undo and redo buttons. Initially, when I added the tooltip text, it appeared directly inside the header instead of inside a floating bubble. This happened because the tooltip elements were not correctly styled or positioned relative to their parent buttons. After adjusting the CSS and adding additional wrapper elements, the tooltips appeared as floating message bubbles. However, the next problem was that they were not aligned correctly with the buttons and sometimes appeared over other UI elements such as the clear button.

To solve this, I implemented a small JavaScript positioning script (tooltip.js) that calculates the button's position using `getBoundingClientRect()` and places the tooltip directly below the hovered element. This allowed the tooltip to stay correctly aligned regardless of where the button appears in the header.

Another issue occurred while implementing the collapse button redesign. When the button was positioned inside the `#toolPanel` container using absolute positioning, it was partially cut off by the panel edge. Even when using `overflow: visible`, the button still appeared clipped due to how the container was structured. I eventually solved this by moving the collapse button outside the panel element in the HTML structure so it became a sibling element instead of a child element. This allowed the button to be positioned independently using `position: fixed`, which removed the clipping issue completely.

I also encountered issues while trying to resize the colour palette interface. Initially I tried to adjust the palette size through CSS rules, but nothing appeared to change. After investigating the project structure, I realised that the palette I was attempting to style was not the one currently being rendered by the application. The visible palette was actually generated dynamically through the tool system, which meant my earlier CSS changes were targeting the wrong element. After identifying the correct component, I adjusted the styling for the slider container instead.

Another minor issue occurred with the tooltip script not appearing to work at all. After debugging the problem, I discovered that the JavaScript file was not loading because it had been placed in the wrong directory. Once I corrected the file path and ensured the script was referenced correctly in `index.html`, the console output confirmed that the code was executing properly.

Overall, although there were several UI-related issues during this stage, I was able to resolve them through debugging, restructuring the HTML layout, and refining the CSS and JavaScript interactions.

What are you planning to do over the next few weeks?

I plan to focus mainly on testing, debugging, and refining the user interface rather than continuing to add large new features.

At this point most of the major drawing tools have already been implemented, including the freehand tool, spray can tool, mirror drawing tool, stamp tool, and eraser functionality. Because of this, my main priority is ensuring that all tools behave consistently and that the interface feels polished when switching between them.

I also want to continue reviewing the application from a usability perspective. For example, I plan to test how easily users can understand the sliders, preview indicators, and button layouts without needing instructions. If any interactions feel confusing, I will make further adjustments to the UI structure.

Another goal is to perform more systematic debugging and testing across different screen sizes, especially since responsive CSS rules have now been added for tablet and smaller displays.

Overall, the next phase of the project will focus on stability, usability, and interface polish rather than introducing entirely new tools.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

Most of the core functionality has already been implemented, and this stage of development has allowed me to focus on improving the user interface and resolving smaller design issues. While these refinements sometimes required additional debugging and restructuring, they helped make the application feel more cohesive and professional.

The main risk at this stage would be spending too much time experimenting with visual design changes instead of ensuring the application is stable. To avoid this, I am now shifting my focus toward testing and debugging the existing tools rather than adding additional features.

By concentrating on polishing the interface, improving responsiveness, and ensuring that each tool works reliably with the undo/redo system and UI controls, I am confident that the project will meet the required functionality and quality standards.